

DAI: A Domain-Specific LLM with Model-Excluded Authority, Capability-Disjoint Verification, and Recomputable Receipts

Hiago Kin Levi
Decripte · hiago@decripte.us

This paper describes DAI (Decripte Artificial Intelligence), a proprietary domain-specific language model within a governed defensive architecture. It explains the model and training loop alongside structured target extraction, declared action contracts, deterministic admissibility, bounded execution, independent observation, sealed receipts, and subsequent review and reconciliation.

Edition	Post-filing preprint prepared 1 October 2026
Manuscript	Technical body preserved from the supplied manuscript
Patent status	U.S. provisional application 64/166,855; patent pending
Publication status	Preprint; not represented as peer-reviewed
Language	English

The reported experiments and deployment observations refer to the version and period described in the manuscript. Formal results depend on their stated assumptions. This edition does not assert that every theoretical construction is implemented or independently validated, or that the software is publicly available.

© 2026 Hiago Kin Levi. All rights reserved. Public availability of this manuscript does not grant a license to proprietary source code, model weights, or patent rights. No third-party scientific endorsement is asserted.

DAI: A Domain-Specific LLM with Model-Excluded Authority, Capability-Disjoint Verification, and Recomputable Receipts

Hiago Kin Levi

Post-filing edition. U.S. provisional application 64/166,855, received 1 October 2026. Patent pending; no patent has been granted. DAI is the proprietary LLM and its governed defensive architecture; ADAI names the proposed technical category.

Abstract

We describe DAI (Decripte Artificial Intelligence), a deployed system in which a software agent conducts an incident-response episode end to end — curating a signal, extracting a target from structured evidence, selecting a reviewed playbook, emitting an action contract, having that contract adjudicated by a decision function that is pure in its declared inputs, executing at a network edge, verifying through a credential provably incapable of writing, sealing a hash-linked transcript, and returning on a schedule to review, reconcile, and undo — with no human approving any step.

The system's contributions as an artifact are: (i) a decision function $\delta(a, \lambda, h)$ whose domain excludes every learned output, making independence from the model a consequence of the signature rather than of a runtime check; (ii) separation of duties realized as **capability disjointness** between two credentials, verified empirically against the enforcement provider's API before deployment — the verifying credential receives an authentication error when it attempts to write; (iii) a receipt satisfying a *recomputation contract*, recomputed in production by a verifier importing none of the system's code, with exact tamper localization demonstrated by controlled alteration; (iv) **per-principal perimeters inside a shared enforcement domain**, requiring containment predicates that conjoin a hostname discriminator with the source address, and an aggregation scheme that multiplies simultaneous-containment capacity by the expression-length limit under a platform rule budget; (v) an **element-identified reconciliation** algorithm that remains sound under that aggregation, where the conventional rule-identified algorithm is unsound in both directions; and (vi) a domain language model trained exclusively on the system's own outcome-verified transcripts, which narrates and is structurally incapable of deciding.

We report production measurements: a containment executed, verified by a distinct credential, and sealed 887 ms after the episode's first turn, including two round trips to the enforcement provider; a receipt of twelve turns recomputed externally with all links valid and, under single-character alteration of turn zero, failing at exactly the altered index; zero-blast-radius isolation confirmed by the absence of any domain-wide rule for the contained address; and the full testimony→review→undo cycle closing. We also report ten defects found in the period, every one of which was silent, and analyse what each revealed.

Keywords: autonomous cyber defense, agentic systems, tenant isolation, capability-disjoint verification, tamper-evident receipts, reconciliation, domain-specific language models.

1. Introduction

1.1 Scope

DAI automates the detection, containment and recovery phases of the standard incident-handling cycle [3]. It defends infrastructure belonging to its operator and to two sibling products of the same organization. It ingests signals from network-edge analytics and from a cloud threat-detection service, opens an episode per material signal, and — where the structured evidence yields an actionable target and a reviewed playbook covers the signal type — changes the network edge to contain the source.

It is **defensive by construction**: the catalogue of expressible actions contains only containment and challenge operations on perimeters its operator controls. There is no reconnaissance, no scanning, no outward-directed action. This is enforced by a build gate that runs before the test suite, on the reasoning that if the guarantee that the system does not attack has been weakened, nothing the suite reports afterwards is relevant.

1.2 DAI is an ADAI, and this is checkable

The companion paper defines **ADAI** — *autonomous defensive artificial intelligence* — not as a capability but as a class with four membership criteria. DAI is an instance of that class. The claim is not rhetorical: each criterion is checkable, and three of the four are checkable without our cooperation.

Criterion	What it requires	How DAI satisfies it	Where verified
A Acts on production	Configuration change to a system carrying real traffic	Creates and removes rules at a network edge fronting three live services	§9, first row
B Decides without contemporaneous approval	For some action class, the determination is the system's	Runs at autonomy level A4; zero episodes await a human	§3.4, §9
C Admissibility adjudicated outside the proposer	The decision function's domain excludes learned output	$\delta(a, \lambda, h)$ — contract, deployment constant, one boolean; no model in the signature	§3.3
D Evidence a non-operator can verify	Integrity decidable from the record alone	Twelve-turn receipt recomputed by a verifier importing none of our code; altered turn localized exactly	§4.4

Criterion **D** is the one we would ask a reader to test first, because it is the cheapest to falsify and the most often claimed without basis. Take a receipt, implement a verifier from its published description, run it, then alter one character and confirm both that the verifier rejects and that the index it reports is the index altered. We report the result of doing exactly that in §4.4, including that the **first** receipt subjected to the test failed (§8.2).

The mechanisms this paper claims — per-principal perimeters in a shared enforcement domain (§5), capability-disjoint verification (§5.4), the recomputation contract (§4.3), and the two consequences of aggregation (§5.7, §6.2) — are set out in those sections, each with the measurement or the proof that supports it.

1.3 Which problems turned out to be hard

Not the ones anticipated. Deciding *what* to do is comparatively easy: the space of sensible responses to a hostile source at a network edge is small, and a reviewed catalogue covers it. The load-bearing problems were:

1. proving to a party that does not trust the system that what it says it did is what happened (§4);
2. making it structurally impossible for one principal's incident to alter another principal's perimeter, in a deployment where all three share one enforcement domain (§5);
3. guaranteeing that no effect created in the world is ever orphaned from its record, including when the process dies between acting and persisting (§6);
4. keeping a learned component in the loop without letting it near the decision (§7).

1.4 Notation

We reuse the notation of the companion framework paper: P principals, $\Pi : P \rightarrow 2^{\mathcal{V}}$ the perimeter map, $\mathcal{G}$ the expressible rule predicates at an enforcement point, $a = \langle \theta, x, \pi, \sigma, \rho, \ell \rangle$ an action, $B(a)$ blast radius, $\mathcal{E}$ the action envelope, δ the decision function, κ the canonical serialization, H the hash, d_i turn digests, D the seal.

2. Architecture

2.1 Layers

The decomposition follows the policy-decision/policy-enforcement separation of access control [22, 25] and the Monitor–Analyse–Plan–Execute staging of autonomic computing [15, 13], with the two additions the companion framework requires: a gate between Plan and Execute whose domain excludes learned output, and a world→records return path.

Layer	Responsibility
domain	value types: autonomy level, effect scope, reversibility
realtime	role and authority contracts; episode event schema
orchestration	curation, evidence extraction, planning, policy, transcript, runner, review, queue, perimeter map
connectors	connector SDK; edge adapters; ingestion adapters; reconciliation
api	HTTP surface, repository, receipt, testimony, signal entry
llm	tokenizer, architecture, corpus, training, narrator, inference
laboratory	findings over the corpus; measures, never changes
apikeyes	system-consumer credentials
entrypoints	processes

Two structural rules are enforced at build time. Layer ownership is declared and checked, so the tree cannot grow into a layer belonging to a sibling system. And the connector SDK, which is duplicated across two codebases by an explicit tracked exception, has the normalized syntax tree of each module

digested and compared, so a logic change on one side alone fails the build — the alternative being that a fix to a connector reaches one of two code paths and nothing signals the divergence.

2.2 Processes

Process	Role
API	HTTP: console, signal entry, receipts, testimony. Reachable through a tunnel; no public port on the host.
Core	Consumes the episode queue and conducts episodes. Holds the edge credentials.
Edge observer	Scheduled: edge analytics → signals.
Threat-feed observer	Scheduled: cloud threat detection → signals.
Reviewer	Scheduled hourly: review, undo, reconcile, close hung episodes.
Narrator	Scheduled: narrates laboratory findings.

The API and Core processes hold **different** execution roles. They shared one until review found that the internet-facing process could consume the episode queue and write to the corpus, including the verified-outcome prefix used as training signal. This is least privilege [24] applied at the process boundary: the process exposed to the network receives the smallest permission set that lets it do its job, and "it is the same application" is not a reason to widen it.

2.3 Assumed adversary

We adopt the companion paper's threat model without change: the adversary authors signal content, chooses timing and volume, may hold a compromised operator account, and observes the system's outputs; the adversary does not hold the executing credential, cannot write below the application to the evidence store or to the seal, cannot compute second preimages, and does not control the enforcement provider.

The last exclusion is the system's most significant unmodelled assumption and is worth restating in concrete terms: verification re-reads the edge through the same provider that executed the change, using a second credential of that provider. A provider that answered both credentials consistently and falsely would produce a **verified** status for a containment that does not exist, and nothing in the architecture would detect it. What the construction does exclude is the far more frequent failure — *the system* being wrong about what it did — which §8 shows is not hypothetical.

2.4 Episode flow

$$\text{signal} \rightarrow \text{curation} \rightarrow \text{hypothesis} \rightarrow \underbrace{\text{ext}}_{\text{evidence}} \rightarrow \underbrace{\text{cand}(\theta)}_{\text{playbook}} \rightarrow a \rightarrow \delta \rightarrow \underbrace{\text{exec}(c_e)}_{\text{executor}} \rightarrow \underbrace{\text{verify}(c_v)}_{\text{verifier}} \rightarrow \text{seal}$$

with a scheduled return path

$$\text{seal} \rightarrow \text{review} \quad \text{and} \quad \text{world} \rightarrow \text{reconciliation} \rightarrow \text{records}.$$

Every arrow writes a turn. The transcript is the primary artifact: an effect without a transcript is treated as a defect to be reconciled (§6), not as a successful action with incomplete logging.

2.5 The twelve mechanisms, and where each one is

The companion paper names twelve mechanisms and binds each to the definition or theorem that gives it meaning. This table is the other half: **where each one lives in this system, and what is measured about it.**

It exists because a framework paper naming twelve mechanisms and an implementation carrying eleven is a gap nobody sees — the twelfth continues to be discussed in prose, and prose reads as description.

Abbr.	Where it is	Measured
AC	Deterministic decision function over the action contract	12 adversarially constructed violations, all refused (EIR = 1)
MEA	The signature of δ : contract, deployment constant, one boolean	A build gate refuses a fourth parameter, and refuses a model SDK outside one confined layer
ACO	The contract object, validated before it reaches the decision point	Constructed violation: maximum autonomy with an empty rollback — refused
EBT	Target extraction over a fixed key set; free text outside the domain	Constructed violation: address named only in an adversary-influenced field — refused. And see §8.9
PPB	Perimeter resolved from the session's principal by strict equality	17 tests over the perimeter map; no default path exists to construct
DSC	Containment predicate conjoining hostname with source address	Zero domain-wide rules created for a contained address
SAV	Two credentials at the enforcement provider, capability-disjoint	The verifying credential receives an authentication error when it attempts to write
SSC	Digest over ten fields including the predecessor's; seal stored separately	12 links recomputed valid; 36 constructed alterations, all detected
PPR	Exported receipt carrying the full recomputation contract	Recomputation rate 1; localization exact on the 23 where a conjunct failed
ELR	Reconciliation keyed on the contained element	Reports three counts — reconciled, orphaned, unverifiable — never two
RBT	Testimony route writing evidence; deterministic verdict reads it	Full testimony→review→undo cycle completed, rule removed and confirmed absent
SRL	Scheduled review opening a new sealed session citing the original	Runs hourly; the original is never reopened

What this table is for, and the one row that is not comfortable

A reader can take any row and ask for the artifact. That is the point: the inventory is finite, so the question *which of the twelve does this system implement* has an answer, and the answer is wrong when it is wrong.

EBT is the uncomfortable row. The guarantee holds by construction — the extraction function's domain excludes free text, and the harness exercises it. What does *not* hold is third-party verification: the transcript records the signal's summary, not the structured evidence the parser read. A recipient of a receipt confirms contract, decision, effect and verification, and must **take our word** for where the target came from.

That is precisely the kind of trust the receipt exists to make unnecessary, and §8.9 reports it as the open defect it is rather than as a limitation of the measurement.

3. The envelope, implemented

3.1 Contract and catalogue

An action is proposed as a contract with declared fields: phase, action type, target type, target reference, parameters, preconditions, postconditions, rollback steps, risk level, reversibility, required autonomy level, effect scope, idempotency key, timeout.

Construction is by instantiation from a catalogue $\mathcal{C}$ of reviewed playbooks — the orchestration model [14] retained deliberately — with $\text{cand} : \Theta \rightarrow 2^{\mathcal{C}}$ mapping signal types to candidate sets. A higher-authority role may express a preference *within* $\text{cand}(\theta)$; it cannot introduce a candidate. The image of the construction is therefore $\bigcup_{\theta} \{\text{instantiate}(c, x) : c \in \text{cand}(\theta)\}$ regardless of any learned output.

The present catalogue admits three signal types and containment actions on address targets only. A signal type outside $\text{dom}(\text{cand})$ produces an episode that investigates and records the gap: absence of a playbook is a legitimate outcome, not an error to route around, and the recorded gap is the laboratory's input for proposing a new playbook to a human.

3.2 Grounded targets

Extraction reads the signal's structured evidence field, parsed as JSON, over a fixed key set. Free-text fields are ignored deliberately: acting on an identifier that appears in adversary-influenced prose is the indirect prompt-injection threat [10, 23, 7] expressed as an effect rather than as an output.

Formally: the actionable kinds are $K_A = \{\text{ip}\}$, while the recognized kinds $K \supsetneq K_A$ include autonomous-system number and country. The latter are recorded as *observed* and never offered, because their blast radius is a provider's entire customer base — frequently including the principal being defended. No envelope predicate over a single principal can bound that, so the correct response is that the target kind never becomes a target.

3.3 The decision function

The decision is

$$\delta(a, \lambda, h) \in \{\text{ALLOW}, \text{DENY}, \text{APPROVE}\},$$

a pure function of the contract a , the deployment ceiling λ , and a boolean h recording the presence of an authorization turn emitted by a role holding authorize authority. h is an **input, not an inference**: the decision point does not derive approval from the contract's contents. The module calls no model; the rules are evaluated most-restrictive first and the first match decides.

Two rules are worth stating because they are refusals the system makes against itself:

$$\ell(a) = A_4 \wedge \rho = \varepsilon \implies \text{DENY}$$

— the contract type permits an empty rollback; the policy does not. Maximum autonomy without a path back is inadmissible regardless of how narrow the scope is.

$$\ell(a) \succ \lambda \implies \text{DENY with recorded reason}$$

— and the reason is recorded as a turn, so a refusal is as auditable as an action.

Because δ 's domain excludes model output, independence from the model is a property of the signature. No runtime check enforces it and none is needed: a model that produced a persuasive justification for an inadmissible action would change nothing, because the justification is not an argument δ accepts.

3.4 Ceiling

Production runs at A_4 . The ceiling is set in the deployment environment; the code carries a default. The two drifted once — an episode narrated "autonomy A_3 above the A_2 ceiling" after the code default had moved to A_4 — so the deployment now asserts the environment value on every rollout rather than depending on recollection.

4. Evidence

4.1 Chain construction

The construction is the classical hash-linked log [11, 27, 5, 18, 17], specialized so that the chaining field lies *inside* the hashed object.

$k = 10$ fields are hashed, in a fixed order: actor role, authority, creation time, kind, model key, model version, payload, previous digest, sequence, episode identifier. Canonical serialization sorts keys, uses separators `,` and `:`, disables non-ASCII escaping, and coerces non-serializable values by a fixed rule.

$$\tau_i.\text{prev} = \begin{cases} \perp & i = 0 \\ d_{i-1} & i > 0 \end{cases}, \quad d_i = H(\kappa(\tau_i)), \quad H = \text{SHA-256}.$$

The previous digest is one of the ten hashed fields, so chaining is *internal* to the serialized object rather than by concatenation. A verifier must therefore reconstruct the object; this is why the exported field list must be complete (§4.3).

The episode row stores the seal D separately, under a database constraint refusing to mark an episode sealed while D is absent. The constraint has fired in production: the system created a real edge rule and then failed to persist the episode, because the insertion path did not set the sealing timestamp. The constraint converted a silent inconsistency into a loud failure — and produced the orphaned effect that motivated §6.

Insertion is the only admissible mutation, enforced by a database trigger below the application. The trigger has been exercised in anger: it refused a migration that would have rewritten the principal identifier of 48 historical episodes. The correct response was to fix forward rather than to bypass it, and we record that the temptation to bypass was real — those 48 episodes carry an identifier corresponding to no principal, and they still do.

4.2 Verification path

Verification does not trust what the write returned. It re-reads through the verifying credential and judges the postcondition:

Let R_p be the rule observed for principal p , X_p its contained set, and μ the contracted mode.

$$\text{status} = \begin{cases} \text{verified} & R_p \neq \perp \wedge x \in X_p \wedge \text{mode}(R_p) = \mu \\ \text{inconclusive} & R_p \neq \perp \wedge x \in X_p \wedge \text{mode}(R_p) \neq \mu \\ \text{failed} & R_p \neq \perp \wedge x \notin X_p \\ \text{failed} & R_p = \perp \end{cases}$$

The two **failed** rows are distinct states with the same verdict and different diagnoses: in the third the perimeter is defended and this target is not in it; in the fourth the perimeter carries no rule of ours at all, which also means the reconciliation of §6.2 will find nothing to adopt. Collapsing them would lose the distinction exactly where an operator needs it.

Absence is **failed**, not **inconclusive**: what was promised was that the containment exist and hold. A different severity than contracted is **inconclusive**: containment happened, but not the contracted one, and calling that either **verified** or **failed** misstates it. The asymmetry is deliberate — **inconclusive** is reserved for *the world disagreeing about degree*, never about existence.

4.3 The receipt

The exported receipt carries: episode metadata; the full chain with all ten hashed fields per turn; highlighted turns (contract, decision, execution, independent verification); **both** the stored seal and the digest recomputed from the chain's tail, separately, so divergence is visible; the algorithm — function, ordered field list, serialization rules, chaining rule; seven numbered verification steps; and an explicit statement of limits.

The limits statement is part of the artifact by design. The receipt establishes *process*: that these turns, in this order, with these contents, were recorded and not subsequently altered; that the contract preceded execution; that verification was recorded under a role distinct from the executor. It does not establish *origin*: that the signal was true, or that the world matched the description — the distinction between record integrity and record veracity that the provenance literature makes explicit [21]. It says so, so that a reader cannot take from it a claim it does not support.

4.4 External recomputation, measured

A verifier written from the receipt's published description, importing none of the system's code, was run against a production receipt of twelve turns:

Chain	Per-turn validity	Tail = seal
unaltered	[True × 12]	True
turn 0 payload altered by one character	[False, True, True, ...]	False

The second row is the behaviour that distinguishes a chain from a checksum: turn 0 fails its own digest and every subsequent turn fails its previous-digest link, so the least failing index is exactly the altered index. Tamper localization is exact.

This test is why §8.2 exists. The first receipt subjected to it **failed**, on every turn, and failed for a reason no internal test could have found.

5. Per-principal perimeters in a shared enforcement domain

5.1 The problem, concretely

Multi-tenant isolation is normally studied on the data and storage planes [16]; here the shared resource is the enforcement point itself.

Three services — the operator's console, a management product, and an offline-capable product — are served from one DNS zone behind one edge network. All three resolve to the same anycast addresses, so all three traverse the same enforcement point.

The mechanism available at the outset was domain-level source access rules: predicates of the form $\text{from}(x)$, independent of destination. By the discriminator lemma, no such rule is tenant-sound: a rule raised because of one product's incident denies service to all three, including the console an operator would use to watch the incident unfold.

5.2 Perimeter bound to principal

The perimeter map Π is declared in code, not in mutable configuration, and resolved by the episode's principal with **strict equality and no default**.

The choice of code over configuration is deliberate and is the same argument used for the system-consumer identifiers: this map decides *where* a containment credential may write. In a mutable store, whoever can write the store can point one principal's perimeter at another's domain — and the credentials themselves live in that store, so the marginal cost of the stricter placement is zero while the marginal benefit is that the change passes review and appears in a diff.

There is no "if the principal is unknown, use the operator's perimeter". That default is precisely how one principal's incident becomes another's outage.

5.3 Hostname-scoped containment

The discriminator is the request's host field. Containment is expressed as

$$g_p = (\text{host}(t) = h_p) \wedge (\text{src}(t) \in X_p), \quad X_p \subseteq \mathcal{X},$$

one rule per principal carrying a set of addresses, rather than one rule per containment.

Capacity. The platform admits L custom rules, of which k were already deployed for unrelated production protections, leaving a budget of $L - k$. Let M denote the greatest number of addresses an expression can hold, and $q = |P_Z|$ the number of principals.

$$\text{cap}_{\text{per-containment}} = L - k, \quad \text{cap}_{\text{per-principal}} = \min(L - k, q) \cdot M, \quad \text{cap}_{\text{shared}} = M.$$

The quantity $N = \sum_p |\mathcal{X}_p|$ is the *demand* — containments currently in force — and a layout is feasible while $N \leq \text{cap}$. Writing the ratio as $N/(L - k)$, as an earlier draft of this section did, compares a demand with a capacity and is meaningless; the correct comparison is

$$\frac{\text{cap}_{\text{per-principal}}}{\text{cap}_{\text{per-containment}}} = \frac{\min(L - k, q) \cdot M}{L - k} = M \quad \text{whenever } q \geq L - k,$$

so aggregation multiplies capacity by the expression-length limit, which in the deployment of §9 is three orders of magnitude. Note also that $\text{cap}_{\text{shared}} = M$ is *smaller* than $\text{cap}_{\text{per-principal}}$ when $q > 1$: the budget-minimal layout is not the capacity-maximal one, because all principals compete for one expression. The shared rule saves rule slots and spends address slots.

Aggregation is not an optimization here; without it the system does not function.

Why one rule per principal, and not one rule for all. The budget-minimal layout is a *single* rule holding every principal's set as a disjunction, $\bigvee_p[(\text{host}(t) = h_p) \wedge (\text{src}(t) \in X_p)]$, which costs one slot rather than $|P_Z|$ and is equally tenant-sound. We do not use it, and the reason is not budget.

Neither layout achieves credential isolation: the provider scopes write tokens to the **domain**, so under either one the executing token for p is technically able to write any rule in the zone. What the per-principal layout does buy is failure isolation of two kinds. A defect that corrupts the expression written for p can destroy only X_p , where under a shared rule it can destroy every principal's containment at once. And the lost update of §5.7 becomes possible only between two episodes of the *same* principal, rather than between any two episodes in the installation.

We therefore spend $|P_Z| - 1$ rule slots to divide the blast radius of a programming error, and the contention domain, by $|P_Z|$. The companion paper gives the general statement; here it is worth noting that the budget-minimal choice is also the one in which a single defect silently disarms the whole installation.

Address ordering. X_p is serialized in sorted order, so the same containment set always yields the same expression text. Without this, verification re-reads a string differing only in order and reports divergence where none exists.

The adapter never writes the whole rule set. The provider's API accepts a replacement of the entire set, and that is the shortest implementation path; a defect there would delete the k pre-existing protections. Only per-rule create, modify and delete operations exist in the adapter, and a test fails any write outside that shape or touching a rule that does not carry the system's mark.

Uniform observation shape. The hostname adapter returns the same observation structure as the domain adapter — rule count, modes, rule identifiers. Deliberately: a second shape would require the runner and the reviewer to know which adapter produced which observation, and that is exactly the information the episode path must not need. The adapters are interchangeable at the seam.

5.4 Credential capability, verified empirically

Each perimeter has two tokens. The executor writes; the verifier reads. Both are scoped to one domain.

Least privilege is verified against the provider's API before the credentials enter production, rather than inferred from permission names. For the hostname-scoped pair, the verifier's attempt to create a rule returned

```
{"success": false,
  "errors": [{"code": 10000, "message": "Authentication error"}]}
```

while the executor created and then deleted a disabled probe rule successfully, leaving the k pre-existing production rules intact.

This is separation of duties [4] realized as capability disjointness [8, 20] rather than as role assignment [25], and it is stronger than a code-path guarantee. "Whoever verifies did not execute" as a property of program structure is broken by a refactor, and broken *silently* — the verification still runs, still returns success, and now proves nothing. As a property of the credential it survives refactoring: a defect that swapped the two adapter instances would produce an authentication error on the write, not a false verification.

5.5 Production evidence of isolation

A signal delivered through the offline product's API key, naming a scanning source, produced exactly one rule:

```
managed_challenge    (mark) · <product hostname>
    (http.host eq "<product hostname>" and ip.src in {<address>})
```

with domain-level source access rules for that address: **0**. The k pre-existing custom rules were intact. The console's hostname was unaffected *structurally* — the expression cannot match it — rather than incidentally.

Isolation was also demonstrated in the negative, before the perimeter existed: a signal from the management product reached the executor and stopped with "no defended perimeter registered for this principal", and the target was not blocked anywhere.

5.6 A correction worth recording

We initially concluded, and reported, that the management product was undefendable at this enforcement point, on the evidence that a plausibly-named host carried a non-proxied DNS record and resolved to a single origin address. Both halves were true; the conclusion was wrong, because that host is not the management product. The product is served at the domain apex, which resolves to the edge network's anycast addresses.

The error is methodologically interesting and we report it rather than quietly fixing it. It began by reading a flag on a host whose name was *assumed* to name the service, and the name was plausible. Every subsequent check confirmed true facts about the wrong host — including an HTTP request, which returns an edge-network server header for that name by an unrelated path. What separates the cases is **resolving the name and comparing against the anycast range**, and that is now the registration criterion. The resolution table is recorded alongside the perimeter map so the next reader need not repeat the measurement.

The correction strengthens §5.3's argument: with the management product at the *apex*, hostname scoping is not a convenience but a necessity, since a domain-level rule raised by its incident would cover the console and the offline product both.

It also exhausts the budget. With L rules, k pre-existing, and two perimeters, $L - k - 2 = 0$. A third perimeter does not fail silently — the provider refuses, the adapter raises, and the episode narrates that it did not contain *after re-reading the edge* — but the system would then detect and watch. Another perimeter requires a different plan, not different code.

5.7 The rule is shared mutable state, and one writer is a deployment accident

Aggregation (§5.3) buys capacity and costs atomicity. Containing x for principal p is read-modify-write over a *set*: read X_p , compute $X_p \cup \{x\}$, write. The provider's per-rule endpoints offer no compare-and-swap precondition, so two episodes containing different addresses on the same host can interleave as

$$\text{read}_1 \prec \text{read}_2 \prec \text{write}_1 \prec \text{write}_2$$

and the second write, computed before the first existed, silently drops x_1 .

The write does not detect it. The provider returns success; the rule was updated exactly as asked. What was asked was computed from stale state.

Independent verification does detect it, without knowing the mechanism: the losing episode re-reads through the verifying credential, finds its target absent from X_p , and records **failed**. This is the third row of §4.2's table arriving from a cause nobody designed it for, which is the argument for structural predicates over targeted checks.

Today the race is nearly unreachable, and that is the problem. The core runs one task and consumes the queue sequentially, so there is one writer — but that is a property of the *deployment topology*, a process count and a loop, not of the adapter. It stops holding the moment the service scales out, and the hourly review task, which also writes to the perimeter, already runs in a separate process. Depending on a task count to not lose containments is depending on the wrong thing.

The adapter therefore checks, after writing, whether the target actually appears in the set, and repeats the read-modify-write **once** if it does not. One retry, not a loop: if the second attempt also loses, verification re-reads and fails, which is the correct outcome, whereas persistence would mask sustained contention on one host — and sustained contention is a signal about the incident rate, not a condition to absorb. Losing twice does not raise: raising would trade a recorded failed verification for an exception, and the episode would lose the verification turn, which is the proof.

This defect was found by writing the companion paper, not by reading the code. It had no symptom, and would have had none until the first day two episodes for the same principal overlapped.

6. Closing the loop

6.1 Review

The reviewer runs hourly. For each containment it gathers three observable facts — rule still present (re-read through the verifying credential), target recurrent since the containment, time elapsed — and applies the verdict function of the companion paper, with non-recurrence window $W = 24$ h.

The verdict is recorded in a **new** episode citing the reviewed episode's identifier, never appended to the original. The original is sealed, and reopening a sealed record to write in it destroys the only property that made its receipt worth anything.

TOO_EARLY opens no episode at all.

Undoing passes through the same path as acting: executor removes, verifier confirms. Treating rollback as "just cleanup" would exempt the most consequential class of action from the guarantees governing the rest.

Per principal. The reviewer iterates over every registered perimeter, using **that principal's** credential pair. Reviewing one principal's containment with another's adapter would re-read the wrong rule space and report the containment absent — yielding **EXPIRED** by mistake and undoing a legitimate containment. The pair therefore travels with the principal rather than being resolved from process configuration.

6.2 Reconciliation

Review traverses records $\rightarrow$ world, which leaves the window between acting and persisting. That window is not hypothetical: it opened in production when the system created a real rule and the episode write failed immediately afterwards, leaving a rule at the edge with no execution record, invisible to review forever.

The world $\rightarrow$ records traversal is the convergence problem of declarative configuration management [2, 12], under a constraint those systems do not have: the record is evidential, so divergence may not be repaired by rewriting the record.

Domain perimeter (rule-identified). Rules carrying the system's mark are listed through the verifier and each checked against the records in three states: reconciled, no episode, no execution record. The third state exists because "does an episode exist?" proved too coarse — the orphaned rule *had* an episode, stuck in an executing state with no receipt. The remedy differs per state, so the name must too.

Hostname perimeter (element-identified). Here id from rules to episodes does not exist: one rule carries $|X_p|$ addresses originating in $|X_p|$ distinct episodes, and the description is fixed per principal, carrying no per-element information. The unit of reconciliation is therefore the address:

```
for p in principals_defended:
    X ← contained_set(c_v(p))          # verifying credential only
    if X = ⊥: defer; continue          # unreadable ≠ empty
    for x in X:
        if ¬∃ episode of p with (contract targeting x ∧ execution receipt)
            ∧ ¬∃ prior adoption of x by p:
            adopt(p, x, mode ← observed_mode(c_v(p), x))
```

Cost: $O\left(\sum_p |X_p|\right)$ existence queries per cycle.

Three details are load-bearing, each corresponding to an observed defect: the read uses the verifying credential, because reconciliation is observation; an unreadable edge defers rather than being treated as empty, since treating it as empty silently concludes "nothing is contained" while treating it as fully orphaned would adopt the world; and the adopted mode is **read from the edge**, not assumed, because recording a default severity that differs from the actual one makes the next review compare what it read against what it recorded and conclude divergence where none exists.

Adoption, not reconstruction. An unrecorded effect produces an episode recording what is observed *now*, whose narration states that it is an observation of current state and explicitly declines to assert the moment of creation. Writing a plausible creation timestamp would place in an append-only evidential record a proposition no one observed. The expiry clock starts at adoption, which errs toward retaining protection rather than removing it early.

Without adoption the orphan is reported every cycle forever, never reviewed and never expiring — recurring noise, which is how a report teaches its reader to ignore it.

6.3 Hung episodes

An episode in a non-terminal state for more than two hours cannot advance on its own: the message that opened it has left the queue, and the signal travelled in that message. It is aborted with the reason written. Two hours is deliberately generous; a genuinely in-flight episode lasts seconds, and aborting one still working is worse than waiting. This also runs per principal.

Leaving them open is not neutral: the console counts stalled episodes, and a population that never closes is precisely the noise that teaches an operator to ignore the console — the alarm-fatigue failure documented in security operations [1] and in clinical monitoring [6].

6.4 Testimony

A recorded classification — false positive, confirmed, context — with the author's identity. It records and never acts. The next review reads it and may reach `FALSE_POSITIVE`, which removes the rule through the ordinary action path.

The full cycle has run in production: a human recorded a false positive, the scheduled review read it, and the rule was removed from the edge and confirmed absent by the verifying credential.

The distinction between evidence and command is a security property. An interface in which a person's assertion directly actuates the perimeter is an interface in which compromising that person's account directly actuates the perimeter.

7. The domain language model

7.1 Constraint

A founder-level constraint requires that no third-party model participate in the cognitive path — reasoning, decision, action-contract generation, narration. The constraint does not reach voice synthesis or recognition, which remain external behind a gateway.

Its motivations are recorded and are not ideological. Third-party weights carry third-party acceptable-use policies, several of which exclude military use, which is disqualifying for the offline product's government customer. And an external API does not exist on a network with no egress, which is where that product runs; local inference must therefore be a property of the design rather than a later addition.

7.2 Construction

Tokenizer (byte-pair encoding [29] trained on the system's own corpus), causal Transformer [30], corpus extraction from sealed transcripts, training with a **per-episode** validation split so that an episode's turns never straddle the split, narration with a deterministic fallback, and inference.

Two properties are structural.

Outcome-verified corpus. Training reads only episodes with verified outcomes. The learning signal is not "what the system said" but "what the system did and what a capability-disjoint credential confirmed". Label quality dominating dataset size is the recurring lesson of applied machine-learning systems work [28]. This is a much smaller and much more expensive dataset than the alternative, and that is the point: a corpus of unverified assertions teaches a model to produce assertions.

Approval gate. A trained model is published only after passing a gate on corpus size and diversity. Until one passes, the narrator falls back to the deterministic rule — and this is the normal state, not an error state.

7.3 What it may and may not do

It narrates. It cannot decide, by §3.3: the decision function's domain excludes model output. A model producing a persuasive justification for an inadmissible action changes nothing about admissibility.

Build-time enforcement matches the claim. A gate parses the abstract syntax tree of every module and refuses imports of third-party model packages — refusing 21 package roots, including pre-trained-weight loaders, on the reasoning that a third-party dependency need not traverse a network: pre-trained weights are the same dependency by another path. It also refuses inference-service clients of the cloud provider, which would be third-party inference without installing anything. It does **not** refuse the tensor and numerical libraries: those are the tools the system's own model is built with, and refusing them would forbid the very thing the constraint mandates.

The gate reads the syntax tree rather than the text because this project's own documents name the providers it declined to use; a textual search would fail the decision not to use them.

7.4 What we cannot claim

We cannot claim the model beats the deterministic baseline on an adversarial harness, because the harness is designed and not built, and because the corpus is small. The binding constraint is not compute: outcome-verified episodes accrue slowly by construction, and the enforcement platform's plan does not expose the per-address edge events that would most enrich them. This is the corpus-scarcity open problem of the companion paper, met in practice.

8. Defects found, and what each revealed

Every defect below was **silent**: the symptom was absence of signal, not presence of error. We report them because in a system of this kind they are the most informative part, and because the argument for the architecture is precisely that it converts silent failures into loud ones.

8.1 Perimeter not bound to principal

The edge adapter was constructed once from process configuration and used for every episode, without consulting the episode's principal. Read isolation was complete and tested; no principal had ever read another's data. The action path had no equivalent check.

Latent while one principal produced signal; reachable the moment API keys were issued to two sibling products — which is what the keys existed for.

What it revealed. Isolation had been audited entirely on the path where auditing is cheap, because every query carries a visible filter, and assumed on the path that changes the world. Auditing the queries would have returned green. The class is general: **a guarantee verified where verification is easy and assumed where it is expensive.**

8.2 A receipt no one could recompute

The receipt reused the console's display projection, which carries five of the ten hashed fields. Recomputation per the receipt's own instructions produced five failures out of five turns.

Every internal test passed, because every internal test constructed the turn shape by hand or through the system's own serialization.

What it revealed. A property of an artifact, tested through its producer, is untested. The remedy has two parts: the test now calls the real projection rather than reconstructing its shape, and conformance is demonstrated by a verifier importing none of the system's code.

8.3 A query for a value the writer never writes

The review query selected turns whose kind literal differed by a prefix from the one the writer persists. SQL does not object to a literal matching nothing, so the lateral join returned the empty set, silently, for weeks.

Two consequences. Executed containments were **never reviewed** — the loop closed only through the adoption branch, which is why the testimony cycle of §6.4 worked and gave false confidence. And the guard against re-adoption used the same literal and was therefore permanently false, so the same effect was re-adopted every hour: production logs show one rule adopted at 20:44, 20:56, 21:56 and 22:55, with "reconciled: 0" each time.

The same class of error existed in two further places: the connector screen's "last containment" read a kind that never existed and was permanently null, and the receipt's execution highlight was never populated (the full chain was always present, so the proof was unaffected; the index pointed at nothing).

No test caught any of it, because all used a storage double, and a double returns what the test specifies — the classic hazard of mock-based testing [19]. The thing that was wrong was the *literal*, visible only against the real store or against the source text.

Remedy. All three now derive the value from a single enumeration, and the review passes it as a bound parameter. A test reads the source and fails any kind literal the writer cannot produce.

What it revealed. A defect in the interface between two components, each individually correct and individually tested. Doubles encode the implementer's belief about the other side; where that belief is the defect, doubles are blindfolds.

8.4 A documented guarantee that was never enforced

A design record asserted that a build gate covered the model layer and refused third-party model SDKs. The gate checked something else entirely. The absence of such SDKs was in fact true — but by discipline, not by enforcement.

Remedy. The gate described in §7.3 now exists.

What it revealed. **Documentation that switches off inspection.** A reader who believes "the gate covers this" stops checking it, and the cost appears not when the statement becomes false but when someone relies on it. The corresponding process rule — every asserted guarantee carries an enforcing check, or states in its own text that it is discipline and not a gate — is now recorded as a decision rather than left as a habit.

8.5 Silent loss of the adversary's address

Two access paths to the same provider's threat findings use different field capitalizations. The reader used one; the delivery path produced the other. Every finding lost the source address silently — and the test passed, because its double mixed both spellings, a shape the provider never emits.

Remedy. Fixtures captured verbatim from the account.

What it revealed. A double that encodes the implementer's belief about an external format, rather than the format.

8.6 Scheduled components running stale code

The deployment pipeline updated two long-running services and no scheduled tasks; each task definition remained pinned to whatever artifact a human last registered. The task closing the containment loop and the task reading the threat feed both lagged the tested tree — so §8.5's fix had been merged, tested, deployed to both services, and never reached the component that used it.

Remedy. One deployment mechanism covers every component that runs, and reports what it changed. Registering a new task revision suffices, since the schedule targets the family rather than a pinned revision.

What it revealed. Divergence between the tested artifact and the running artifact, along a path no test observes.

8.7 A lost update with no symptom

Containment under aggregation is read-modify-write over a set with no compare-and-swap available, so two overlapping episodes on the same principal can lose one containment; the write reports success, and only independent verification notices. §5.7 gives the mechanism and the mitigation.

What it revealed. Two things. First, that a decision taken for one reason — aggregating addresses into one rule to fit a rule budget — carries consequences in an unrelated dimension, and that the second consequence is easy to miss precisely because the first one is the interesting one. Second, that "there is only one writer" was true and was not a property of this component: it was a process count. A guarantee that holds because of how something is deployed is a guarantee that disappears in a scaling decision taken by someone who never read this code.

How it was found. By writing the formal account of aggregation for the companion paper and noticing that the same premise yields a second theorem. It had no symptom and would have had none until the first overlap.

8.8 The receipt does not authenticate whose session it is

The digest covers ten fields of each turn. The principal is not one of them: the turn carries the session identifier, and the session record carries the principal. A third party recomputing a receipt per §4.4 therefore confirms *what happened* and cannot confirm *on whose behalf*.

The binding is real but local — an append-only constraint on the store — and a local guarantee is worth nothing to an external reader, who has no access to the store and no way to observe that the constraint exists. In a system whose strongest claim is per-principal isolation (§5), that is the wrong thing to leave outside the authenticated set.

What it revealed. That "the receipt proves X" and "the system enforces X" are different sentences, and that the second was quietly being read as the first — by us. The append-only constraint had been treated as if it made attribution part of the evidence, and it does not; it makes attribution part of the *operator's* trust in their own database.

Status: not fixed. Adding the principal to the hashed field set requires recomputing every historical digest, which is precisely what an append-only store exists to prevent — it is a fork of the chain, not an edit. Binding the session record into turn zero's payload fixes it forward, leaves history alone, and is what we intend to do. We report it unfixed rather than describe the intention as the state.

8.9 Target validation narrower than the predicate it discharged

Target validation rejected addresses that are private, loopback, link-local, multicast, reserved or unspecified — the addresses that are not routable sources — and was taken to discharge the bounded-effect-scope predicate. It admitted addresses inside the **enforcement provider's own published ranges**. Two containments against the provider's shared proxy addresses stood in the zone until found by inspection.

Nothing in the reasoning path erred, which is what makes this defect worth the space. The evidence was structured; extraction was sound; the proposed target was the address the evidence named, and under a proxied path the observed source *is* the proxy. The action was still outside the predicate: a shared proxy address has blast radius far above one, since the effect falls on every principal arriving through it — including the defended principal — and it is additionally the path by which the containment would be verified, so the effect and its verification become mutually dependent.

What it revealed. That a check can exist, be correct about what it checks, and be a proper subset of the predicate it is credited with enforcing. §8.4 was documentation asserting a check that did not exist; this is the mirror image, and the harder one to see: the implemented subset is exactly the subset a reviewer thinks to look for. It is also why the envelope-integrity gate attacks the predicates rather than reading the code that implements them — the defect had been readable in the source for as long as the source existed.

Status: fixed. The provider's published ranges are enumerated as static, dated data on the decision path, with a tool that refreshes them by reviewed diff. Static and not resolved at decision time: a live lookup would make δ depend on a network service and cease to be pure in its declared inputs, which would break both §3.3 and the recomputation contract. The case is now in the envelope harness, where its absence had been invisible.

The two standing containments are a separate matter from the defect: removing them is an action, and it goes through review and undo like any other, not through an operator editing the provider's configuration behind the system's back.

8.10 A guarantee that holds by construction and cannot be checked from the receipt

Evidence-Bound Targeting holds. The extraction function's domain is a fixed key set over structured evidence; free-text fields are not in it, and a constructed attack naming an address only in an adversary-influenced field is refused.

What does not hold is third-party verification of it. The transcript's curation turn records the signal's summary, severity, type and source reference — not the `raw_content` the parser read. Checked on an exported production receipt: the contained address appears in the hypothesis summary, in the action contract and in narration, and **nowhere in any evidence field**, because no turn carries one.

The consequence is exact. A recipient of a receipt recomputes the chain, confirms that a contract preceded a decision which preceded an effect which was verified by a disjoint credential — and then must **take the emitter's word** for where the target came from. That is the one thing the receipt exists to make unnecessary.

What it revealed. That *the guarantee holds* and *the guarantee is provable to a stranger* are different claims, and a system can satisfy the first while the second quietly does not apply. The distinction is invisible until something evaluates the receipt against the mechanisms rather than only against itself: the chain verifies perfectly either way, because the missing field was never hashed — it was never there.

Status: not fixed. The repair is to record the structured evidence, or a digest of it, in the curation turn — bringing it inside the hashed field set so the target's provenance becomes recomputable. This is a change to what is hashed, and therefore to the recomputation contract, which is why it is reported rather than applied: the contract is versioned, and historical receipts must keep verifying under the version they were sealed with.

Until then, EBT is reported as *holding by construction and not verifiable from the record* — which is what it is, and is more useful than either of the two comfortable simplifications.

9. Production measurements

Single deployment, three services behind one enforcement domain, over a period of days. These are instances, not statistics.

Observation	Result
First real containment	Rule created at the edge; verified by a capability-disjoint credential; episode sealed
Episode duration, first turn to last	887 ms (12 turns, two provider round trips)
Receipt recomputed externally (12 turns)	all links valid; tail = seal
Receipt with turn 0 altered	fails at index 0 and thereafter; localization exact
Isolation, hostname perimeter	1 hostname-scoped rule; 0 domain-level rules for the same address
Isolation, negative case	signal from an unregistered principal: no action, reason recorded
Testimony → review → undo	closed; rule removed and confirmed absent
Orphaned rule	adopted; entered the normal review cycle
Hung episodes	closed with chains intact
Episodes awaiting a human	0
Episodes in the operator's tenant	114 (81 terminated without action, 33 sealed)
Refusal turns, 30-day window	93, of which 79 (85%) under one criterion
Signals whose evidence never yielded an actionable target	10
Containments with a reviewed outcome	4 (1 false positive, 3 absent)
Post-fix review pass	per-principal; hostname perimeter re-read through its own adapter; verdict <code>TOO_EARLY</code> , reason recorded

The corpus figures come from the system's own findings layer (§10) and its session listing, not from an analyst querying the database — which is the intended use of that layer and is itself a result: the measurement that most changed our account of the corpus (reported there) was produced by the component built to measure without being able to change anything.

The duration is measured between the creation timestamps of the first and last turns, which are written independently. It is **not** measured between the episode's `opened_at` and `sealed_at` fields: those two are written from a single clock read and are identical to the microsecond, so their difference measures nothing. We note this because a field pair that looks like an interval and is not one is the kind of thing that silently becomes a published latency figure — and because the same shape appears in §8 four times under different names.

Build discipline: twelve gates precede image construction — tests, lint, independence, defensive limit, envelope integrity, terminology, contract page, connector-SDK divergence, console egress, credential presence, layer boundary, and API-contract-versus-code — and two deploy. The suite is 622 tests across 50 files, each file executed in its own process, because several substitute module functions to simulate failure and a badly undone substitution in a shared process makes results depend on file order.

The envelope-integrity gate is the one whose failure is not a regression in quality. It constructs violations of each envelope predicate and requires that every one be refused; a single admission means that predicate is not enforced, and the correct response to a red result there is to stop, not to triage.

Two gates encode a lesson rather than a style. The defensive-limit gate runs **before** the tests: if the guarantee that the system does not attack has been loosened, nothing the suite reports matters. The contract gate parses routes out of the application's syntax tree and compares them against the published

API description in both directions, because a contract that drifts from the code is a contract that lies to whoever integrates against it.

10. What is not built

An inventory of capability written by its builders is worth nothing without its complement.

- **Adversarial harness with blind holdout.** Designed, not built. Until it exists, no claim about policy quality is supported, and §7.4 stands.
- **Reconciliation against a world of foreign granularity.** The hostname case is solved (§6.2); the general problem is not.
- **Per-address edge events.** Unavailable on the enforcement platform's current plan, and the binding constraint on corpus diversity.
- **Versioned infrastructure.** Several scheduled task definitions and one execution role exist in the account without a versioned declaration; the provisioning script lags the tree.
- **A third perimeter.** Exhausted by rule budget, not by design (§5.6).

This inventory perishes quickly. Between the first draft of these notes and their first revision, on the same day, the system executed its first real containment and a sentence stating that none had occurred stopped being true.

11. Conclusion

DAI acts on production infrastructure without a human approving each action, and the argument that this is acceptable is not that its model is good. It is that the decision is made outside the model by a pure function of a declared contract; that the credential which verifies is provably incapable of executing; that the perimeter it may write to is bound to the principal it acts for, with no default; that per-principal containment is expressed by conjoining a discriminator with the source, so blast radius is one by construction; that every effect is reachable from a hash-linked record a third party can recompute without the system's code; and that every effect is revisited, reconciled, and reversible.

The defects of §8 are the best available evidence for that structure, because every one was silent and every one was silent for the same reason: the guarantee lived in convention rather than in construction. The system's claim on attention is not that it does not fail. It is that it fails loudly — fail-stop rather than fail-silent [26, 9].

Limitations

Production measurements come from a single deployment over days; they are instances, not statistics. The claims of §3.3 and §5.4 are architectural and hold for the system as constructed. The external recomputation of §4.4 was performed for one receipt and one controlled alteration; it demonstrates the

property, not its frequency. No claim is made about policy quality, which is unmeasured (§7.4), and none about sufficiency for safety — only that each named component is necessary for the guarantee it names.

References

- [1] Alahmadi, B. A., Axon, L., Martinovic, I. (2022). *99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms*. Proceedings of the 31st USENIX Security Symposium.
- [2] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM 59(5), 50–57.
- [3] Cichonski, P., Millar, T., Grance, T., Scarfone, K. (2012). *Computer Security Incident Handling Guide*. NIST Special Publication 800-61 Revision 2.
- [4] Clark, D. D., Wilson, D. R. (1987). *A Comparison of Commercial and Military Computer Security Policies*. IEEE Symposium on Security and Privacy, 184–194.
- [5] Crosby, S. A., Wallach, D. S. (2009). *Efficient Data Structures for Tamper-Evident Logging*. Proceedings of the 18th USENIX Security Symposium.
- [6] Cvach, M. (2012). *Monitor Alarm Fatigue: An Integrative Review*. Biomedical Instrumentation & Technology 46(4), 268–277.
- [7] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352.
- [8] Dennis, J. B., Van Horn, E. C. (1966). *Programming Semantics for Multiprogrammed Computations*. Communications of the ACM 9(3), 143–155.
- [9] Gray, J. (1986). *Why Do Computers Stop and What Can Be Done About It?* Symposium on Reliability in Distributed Software and Database Systems, 3–12.
- [10] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M. (2023). *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 79–90.
- [11] Haber, S., Stornetta, W. S. (1991). *How to Time-Stamp a Digital Document*. Journal of Cryptology 3(2), 99–111.
- [12] Hanappi, O., Hummer, W., Dustdar, S. (2016). *Asserting Reliable Convergence for Configuration Management Scripts*. OOPSLA, 328–343.
- [13] IBM (2006). *An Architectural Blueprint for Autonomic Computing*, 4th edition. IBM White Paper.
- [14] Islam, C., Babar, M. A., Nepal, S. (2019). *A Multi-Vocal Review of Security Orchestration*. ACM Computing Surveys 52(2), Article 37.
- [15] Kephart, J. O., Chess, D. M. (2003). *The Vision of Autonomic Computing*. IEEE Computer 36(1), 41–50.
- [16] Krebs, R., Momm, C., Kounev, S. (2012). *Architectural Concerns in Multi-Tenant SaaS Applications*. 2nd International Conference on Cloud Computing and Services Science (CLOSER), 426–431.

- [17] Laurie, B., Langley, A., Kasper, E. (2013). *Certificate Transparency*. RFC 6962, IETF.
- [18] Merkle, R. C. (1988). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87, LNCS 293, 369–378.
- [19] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [20] Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University.
- [21] Moreau, L., Groth, P. (2013). *Provenance: An Introduction to PROV*. Morgan & Claypool.
- [22] OASIS (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard.
- [23] Perez, F., Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*. arXiv:2211.09527.
- [24] Saltzer, J. H., Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1278–1308.
- [25] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., Youman, C. E. (1996). *Role-Based Access Control Models*. IEEE Computer 29(2), 38–47.
- [26] Schlichting, R. D., Schneider, F. B. (1983). *Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems*. ACM Transactions on Computer Systems 1(3), 222–238.
- [27] Schneier, B., Kelsey, J. (1999). *Secure Audit Logs to Support Computer Forensics*. ACM Transactions on Information and System Security 2(2), 159–176.
- [28] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D. (2015). *Hidden Technical Debt in Machine Learning Systems*. Advances in Neural Information Processing Systems 28.
- [29] Sennrich, R., Haddow, B., Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, 1715–1725.
- [30] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30.

Authorship

Hiago Kin Levi

Brooklyn, NY — 16 September 2026, 22:00

SHA-256 of the Markdown source, to the marker line below:

`ae4fb58e04bd2d2137a6cd2b6f1f9cd9255961bb443217a5fe3dfe849d1f247e`